Daily Scholar Papers Report — 2026-05-28

2026-05-28

Daily Scholar Papers Report — 2026-05-28

Download PDF

Window covered: 2026-05-27 → 2026-05-28 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

Two flagged-author alerts and one Scholar-Recommended batch in the 24-h window. Both flagged papers land in the **fuzzing / model-based testing** axis from opposite ends of the analysis stack: a CAV ’26 paper from Qingkai Shi’s group on *inferring sound and precise symbolic finite automata* for stateful systems, and an ICST ’26 paper from Andreas Zeller’s group on *unifying input + execution constraints* inside the Fandango framework. The Scholar-Recommended bucket carried six papers: three were summarised as incremental fuzzing variants worth a one-line read, and three deep-learning-for-vulnerability-detection papers were screened out under the saturation rubric. No user-curated self-emails were forwarded today.

Outstanding: 1 · **Keep:** 1 · **Borderline High-Priority:** 0

Highlighted Papers

#	Title	Authors	Venue	Link
1.1	Sound and Precise Symbolic Automata Model for Stateful Software Systems (Seal)	Wu, Zhou, Yao, Shi	CAV ’26	PDF
2.1	Combining Input Constraints with Execution Goals (FANDANGO GREY)	Bettscheider, Smytzek, Zeller	ICST ’26	PDF

---

1. Outstanding

1.1 · STATIC ANALYSIS · Sound SFA inference in minutes, lifting dynamic-model-checker coverage 1.6×–3.4× over the prior state of the art

1.1 Sound and Precise Symbolic Automata Model for Stateful Software Systems — Wu, Zhou, Yao, Shi (NJU + ZJU), CAV '26

Verifying stateful systems — parsers, protocol endpoints, embedded controllers — almost always requires a behavioural model, but constructing that model by hand is the dominant cost of model-based verification. *Seal* automates this with abstract interpretation: it derives a **symbolic finite automaton (SFA)** directly from the source code with a *formal soundness guarantee* and a precision that holds up against ground-truth automata.

Definitions. Following the paper's §2 formalisation, an SFA is a tuple `(Q, q0, F, Ψ_D, Δ)` with finite state set `Q`, start state `q0`, accept set `F ⊆ Q`, a set `Ψ_D` of first-order predicates over an input domain `D`, and guarded transitions `Δ ⊆ Q × Ψ_D × Q` (Definition 1). Seal targets the standard *soundness* notion that the inferred automaton's accepted language must contain the implementation's accepted language (Definition 4), and improves *precision* in the strict sense `LM1 ⊆ LM2` (Definition 5). Critically, Seal is the only entry in its qualitative comparison (Table 1 of the paper) with ✓ **Soundness Guarantee × Minutes Scalability × Almost-None Assumptions** simultaneously — Sigma* and FSMExtractor lack the soundness guarantee, and Proteus pays for similar precision with Hours-scale path-explosion timeouts.

Approach. The pipeline runs in three algorithms (numbered as in the paper):

- *Algorithm 1* builds a *guarded-environment automaton* by abstract interpretation: states correspond to path-set partitions of the code, transitions carry abstract environments. Lemma 1 gives the soundness invariant for the iteration step.
- *Algorithm 2* refines the partition so that each automaton state represents a *disjoint* path set. Lemma 2 shows partitioning preserves Lemma 1; Lemma 3 bounds the state count by the number of code locations; Lemma 4 shows widening preserves soundness. Theorem 1 then concludes that Algorithm 1 terminates and produces a sound guarded-environment automaton.
- *Algorithm 3* converts the guarded-environment automaton to an SFA by translating per-transition environments into path-conditions over inputs. Theorems 2–3 transfer the soundness guarantee through this conversion.

Mixed abstract domain (§3.3). The pivotal precision idea is *per-variable* domain selection. The analysis runs in a *precise symbolic* domain by default — keeping exact algebraic expressions over time-indexed input symbols (with negative indices for stream look-back). When a particular state transition fails to converge under the fixed-point computation, Seal demotes *only the offending variables* to a classical abstract domain (intervals, optionally Boxes or Octagons) so the standard widening operator can drive convergence. Variables unrelated to the convergence trouble remain symbolic. The paper contrasts this with reduced products (uniform combination across all variables), trace partitioning (adapts at the trace level), and CEGAR (refines globally on counterexample); Seal's per-variable demand-driven adjustment is the lever that lets a small set of misbehaving counter variables avoid corrupting the symbolic guards of the rest of the automaton. The ablation in Fig. 10 of the paper shows that dropping the symbolic-domain layer and running interval-only collapses median similarity well below the headline 0.95.

Headline numbers (§5). Across 12 real-world subjects spanning message parsers and autopilot stacks (ArduPilot variants — MORPHEUS, SmartRTKL, ArduFlight, ZigZag, ROS, AVTI, RotorTM, etc.), Seal completes inference in minutes; similarity to ground-truth SFAs sits at **0.90–0.99, median 0.95**, against Sigma* and FSMExtractor's **0.11–0.83, median 0.48**. Proteus times out on more than half the subjects. When Seal-inferred SFAs are fed to BooFuzz (for message parsers) and PGFuzz (for autopilots) under a three-hour budget per subject, statement coverage is **1.6×–3.4×** the coverage achieved with

FSMExtractor's SFAs. Ten previously unknown bugs were uncovered during the dynamic-model-checking evaluation.

Why it matters. This is the first SFA-inference tool to combine *minutes-scale latency*, *almost-none implementation assumptions*, and *a formal soundness guarantee* on real-world code. The per-variable mixed-domain trick is the most transferable contribution — directly relevant to taint tracking, summary-based pointer analysis, and any specification-mining loop that wants to keep input symbols algebraically structured for as long as possible before widening. The downstream coverage lift in BooFuzz / PGFuzz also closes a useful loop: model-based testing pays off in practice *when the model is sound*, and that's exactly the property the prior baselines didn't ship.

Tool / artefact at https://doi.org/10.5281/zenodo.20133756.

2. Keep

2.1 · FUZZING · One spec language, five fuzzing paradigms — blackbox, coverage-guided, directed, performance, memory — unified inside Fandango

2.1 Combining Input Constraints with Execution Goals — Bettscheider, Smytzek, Zeller (CISPA), ICST '26

FANDANGO GREY extends the Fandango grammar-+-Python-constraint fuzzer with **execution constraints**: primitives that refer to runtime features of the program under test (basic-block coverage, distance to a target, heap-allocated bytes, execution path length) rather than only to grammar elements. The result is a single specification language in which a tester can write *any* mix of input constraints and execution goals — "I want an input that is valid, as short as possible, and reaches a maximum of code coverage" — and the same evolutionary loop will satisfy them.

Catalogue of execution metrics (Table I). The paper ships a fixed primitive set spanning four fuzzing styles:

- *Directed greybox:* `DistanceToFunction(m, f)`, `DistanceToLine(m, l)`, `DistanceToBB(m, bb)`.
- *Coverage-based:* `FunctionCoverage()`, `CodeCoverage()`, `CoverageInFunction(f)`.
- *Performance:* `ExecutionPathLength()`, `PeakBasicBlockHits()`.
- *Excessive memory:* `HeapAllocatedBytes()`.

These are exposed to the testers as Python-callable values inside the same constraint syntax that Fandango already uses for input properties.

Soft constraint scoring (Algorithm 1). The technical core is how the framework reduces wildly heterogeneous fitness signals — path length, heap bytes, coverage count — to a normalised score that the evolutionary loop can compare. From the paper:

```
function sS(c, t, contrast = 10.0):
    absFitness = c.FITNESS(t)
    cdf        = c.tdigest.CDF(absFitness)
    c.tdigest.UPDATE(absFitness)
    if c.optimizationGoal == "maximize":
```

```
        return exp(contrast * (cdf - 1))
    else:
        return 1 - exp(-contrast * cdf)
```

The t-digest streaming-quantile structure maintains a compact summary of every prior fitness observation; the CDF maps the current value to its empirical quantile in `[0, 1]`; exponential scaling with `contrast = 10` emphasises improvements near the optimisation boundary while keeping early outliers from dominating the budget. The construction lets multiple soft constraints (e.g. “minimise path length” + “maximise heap bytes”) compose additively against a common scale.

Implementation (§IV). A drop-in compiler replacement `fcc` (built on `wllvm` + `extract-bc` + `llvm-link`) instruments every basic block with a unique ID and emits per-run JSON execution traces via an `atexit()` hook. A companion analysis pass `fan` operates on the whole-program LLVM bitcode to build an interprocedural CFG annotated with module / function / line metadata, exposing the static information Fandango needs for distance metrics.

Evaluation (§V). Five domain-specific scenarios:

- *Performance fuzzing* on lunasvg with SlowFuzz-style `ExecutionPathLength()` minimisation — uncovered stability issues at 10-second-bounded inputs.
- *Excessive memory fuzzing* in the spirit of FuzzFactory’s “mem” application.
- *Single-input coverage maximisation* on graphviz, lunasvg, libxml2 — combined input-spec-+-execution-feedback outperforms either alone across all three subjects.
- *Directed greybox fuzzing* on cjson with a 6-key cascade reaching `f234()` only after passing through five intermediate keys — combined configuration reaches the target in **7/10** runs, input-spec-alone in **1/10**, execution-feedback-alone in **0/10**.
- *Population evolution* by rewriting `<start> ::= <input>{K}` with K=10 and maximising `union of CovBBs(<input>)` — cumulative coverage exceeds single-input coverage on every subject.

Caveat. The authors explicitly opt out of head-to-head comparison against AFL, AFLGo, FuzzFactory, etc., justifying this by implementation-language asymmetry (Python vs. C/C++) and the fact that the seed / dictionary assumptions of those tools differ from FANDANGO GREY’s grammar-+-constraint setup. This is methodologically defensible but does cost the paper the absolute speed datapoint a reader might want; the contribution claim is *control and versatility*, not throughput.

Why it matters. The unification is the meat: a single constraint language that subsumes directed / coverage / performance / memory fuzzing collapses the integration cost of running multiple paradigms against the same target. The cjson directed-fuzzing result (7/10 vs 1/10 vs 0/10) is a clean separation of configurations, and the t-digest soft-scoring mechanism is the most readily transferable component — any constraint-aware fuzzer that needs to compose heterogeneous fitness signals can lift it directly.

3. Brief Summaries

3.1 · FUZZING · General-purpose coverage-guided framework targeting compilers, runtime engines, and constraint solvers

3.1 CoverFuzz: A Coverage-Guided and General-Purpose Fuzzing Framework — Lin, Wang, Xu, Long, Zhang — *Journal of Software: Evolution and Process*, 2026

A framework-level proposal aimed at programs whose inputs *are* programming languages — the Csmith / YarpGen / Fuzzilli niche. The contribution claim is generality across SUT classes (compilers, runtime engines, constraint solvers) under a single coverage-guided harness, rather than a new mutation operator. Treated as an awareness item; the deep-read budget is better spent on the flagged-author papers above.

3.2 · FUZZING · AFLNet-style protocol fuzzer with finer state granularity and gradient-guided mutation

3.2 UFG-AFLNET: A Greybox Fuzzing Framework with Fine-Grained State Modeling and Gradient-Guided Mutation for Network Protocols — Xu, Chen, Xin, Yu, Bai, Li — *IEEE TNSM*, 2026

An incremental refinement of the AFLNet / SGFuzz / StateAFL line: finer state modelling than AFLNet's response-code-derived states, plus a gradient-guided mutation operator. Solid journal venue (TNSM) without flagship-security weight; the transferable component is the gradient-guided mutator.

3.3 · FUZZING · Multi-target directed greybox fuzzing for commit-level vulnerability hunting

3.3 Comprehensive Commits Security Testing with Enhanced Multi-target Directed (fuzzer) — Wang, Feng, R. Li — SecureComm 2025 / Springer LNICST

A commit-aware directed greybox fuzzer addressing the standard weakness of single-target DGFs (AFLGo / Beacon / Hawkeye) when a commit changes many direct *and* dependent regions at once. Evidence that "commit-level DGF" remains an active sub-niche through 2025–2026; methodology incremental over the multi-target DGF thread (DAFL / RDFuzz).

Cross-Paper Synthesis

The two flagged papers attack the same workflow — *make fuzzing smarter by sharing structure with semantics* — from opposite ends of the analysis stack. Seal sits on the static-analysis side: it **infers** a sound + precise behavioural model (SFA) of a stateful system so a downstream dynamic-model-checker can drive deep states with valid input *sequences*. FANDANGO GREY sits on the dynamic-analysis side: it **consumes** a grammar + Python-constraint spec and layers execution-feedback constraints on top so a single evolutionary loop optimises for whichever fuzzing goal the tester writes down.

Read together, they sketch an "infer-then-search" workflow that has been a recurring theme across the last fortnight of reports. Seal automates the "spec" half of that workflow for stateful systems; FANDANGO GREY automates the "search" half once a spec exists. A concrete bridge worth prototyping: re-express a Seal-inferred SFA as a Fandango grammar in which state-transition predicates become Python constraints over consecutive `<input>` siblings. ArduPilot variants appear in both evaluations, which makes the experimental subject obvious.

The three "summary" fuzzing papers all live in the *incremental tweak of an existing baseline* sub-genre — pick AFL / AFLNet / AFLGo, bolt on a coverage-guided component, a state-model component, or a multi-target distance, run on a familiar subject set. The contrast with Seal (which contributes a theorem) and

FANDANGO GREY (which contributes a unification) is sharp: both flagged papers are at the *framework* tier, not the *tweak* tier.

A standing meta-observation: deep-learning vulnerability detection — three of today's recommended-bucket papers — continues to publish at A-tier journals but has reached its saturation ceiling, where venue lift no longer compensates for diminishing methodological novelty. The flagged-author / top-conference papers above remain the place to spend deep-read attention.

Writing & Rationale Insights

Seal demonstrates two prose patterns worth absorbing. First, the **qualitative-comparison table as the abstract's structural anchor**: Table 1 lists four approaches across four columns (Assumptions, Soundness Guarantee, Scalability, Class), and the table — not the abstract paragraph — is what the reader uses to slot Seal against its baselines. When a paper has three or more strong competitors, a four-column qualitative table on the first or second page is worth more than two paragraphs of prose-comparison. Second, the use of **Remark-style asides** to forestall obvious referee questions: Remark 1 admits that SEFA loses many of SFA's closure properties, explains why this doesn't bite (decidability is not used in the analysis), and moves on. That keeps the main argument unbloated while still defusing the obvious objection.

FANDANGO GREY illustrates a defensible-but-costly methodological choice: the **deliberate refusal** to compare head-to-head against AFL / AFLGo / FuzzFactory, justified by the difference in implementation language and seed-quality assumptions. The trade-off here is reviewer trust. The unification framing is genuinely a methodological generalisation, but without a single side-by-side speed datapoint — even an unfavourable one on a stripped-down subject — the reader is left to take the qualitative comparison on faith. The transferable lesson for our own tool-paper writing: if you want the "unification across paradigms" framing, supply at least one speed-controlled micro-benchmark so the reviewer has something concrete to anchor against.

Cadence note: this is a representative weekday batch — two flagged-author deep-reads, three summaries, three saturated-skip — without the empty-window pathology of the early-week runs. Infrastructure passed STEP 0–2 cleanly.